



A Guide to Stripe Integration in Spring Boot

As digital transactions rise, the ability to seamlessly integrate payment gateways has become a critical skill for developers. Whether for marketplaces or [SaaS products](#), a payments processor is crucial to collect and process user payments.

This article explains how to integrate [Stripe](#) within a [Spring Boot](#) environment, how to set up subscriptions, offer free trials, and build self-service pages for your customers to download their payment invoices.

What Is Stripe?

[Stripe](#) is a globally renowned payment processing platform available in [46 countries](#). It is a great choice if you want to build a payment integration in your web app owing to its large reach, reputed name, and detailed documentation.

Understanding Common Stripe Concepts

It's helpful to understand some common concepts that Stripe uses to coordinate and carry out payment operations between multiple parties. Stripe offers two approaches for implementing payment integration in your app.

You can either embed Stripe's forms within your app for an in-app customer experience ([Payment Intent](#)) or redirect customers to a Stripe-hosted payment page, where Stripe manages the process and lets your app know when a payment is successful or failed ([Payment Link](#)).

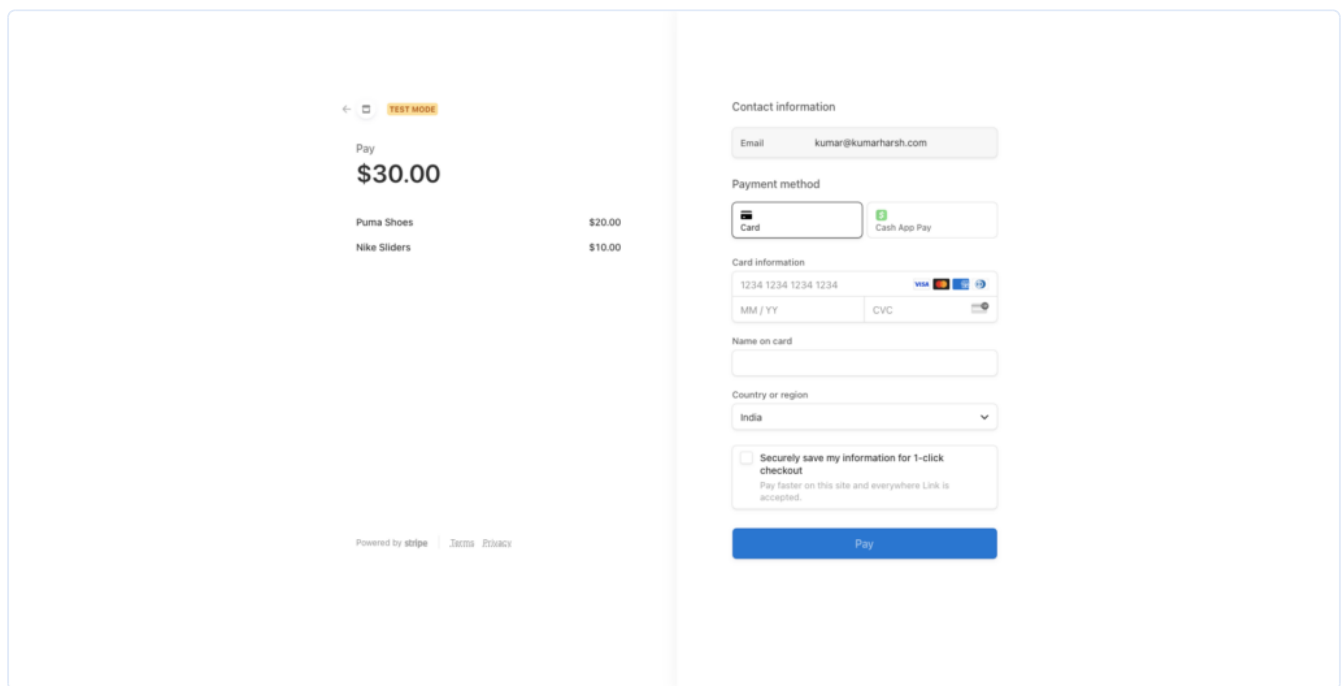
Payment Intent

When handling payments, it's important to gather customer and product details upfront before prompting them for card information and payment. These details encompass the description, total amount, mode of payment, and more.

Stripe requires you to collect these details within your app and generate a `PaymentIntent` object on their backend. This approach enables Stripe to formulate a payment request for that intent. After the payment concludes, you can consistently retrieve payment details, including its purpose, through the `PaymentIntent` object.

Payment Links

To avoid the complexities of integrating Stripe directly into your codebase, consider utilizing [Stripe Checkouts](#) for a hosted payment solution. Like creating a `PaymentIntent`, you'll create a `CheckoutSession` with payment and customer details. Instead of initiating an in-app `PaymentIntent`, the `CheckoutSession` generates a payment link where you redirect your customers. This is what a hosted payment page looks like:



The screenshot displays the Stripe-hosted checkout page, which is divided into two main sections. The left section, titled "Pay", shows a total amount of \$30.00. Below this, a list of items is shown: "Puma Shoes" for \$20.00 and "Nike Sliders" for \$10.00. The right section contains a form for "Contact information" with an email field (kumar@kumarharsh.com). Below this is the "Payment method" section, which includes a "Card" button and a "Cash App Pay" button. The "Card information" section includes fields for the card number (1234 1234 1234 1234), the expiration date (MM / YY), the CVC, and the name on the card. There is also a dropdown for "Country or region" set to "India". At the bottom of the form, there is a checkbox for "Securely save my information for 1-click checkout" and a "Pay" button.

— The Stripe-hosted checkout page.

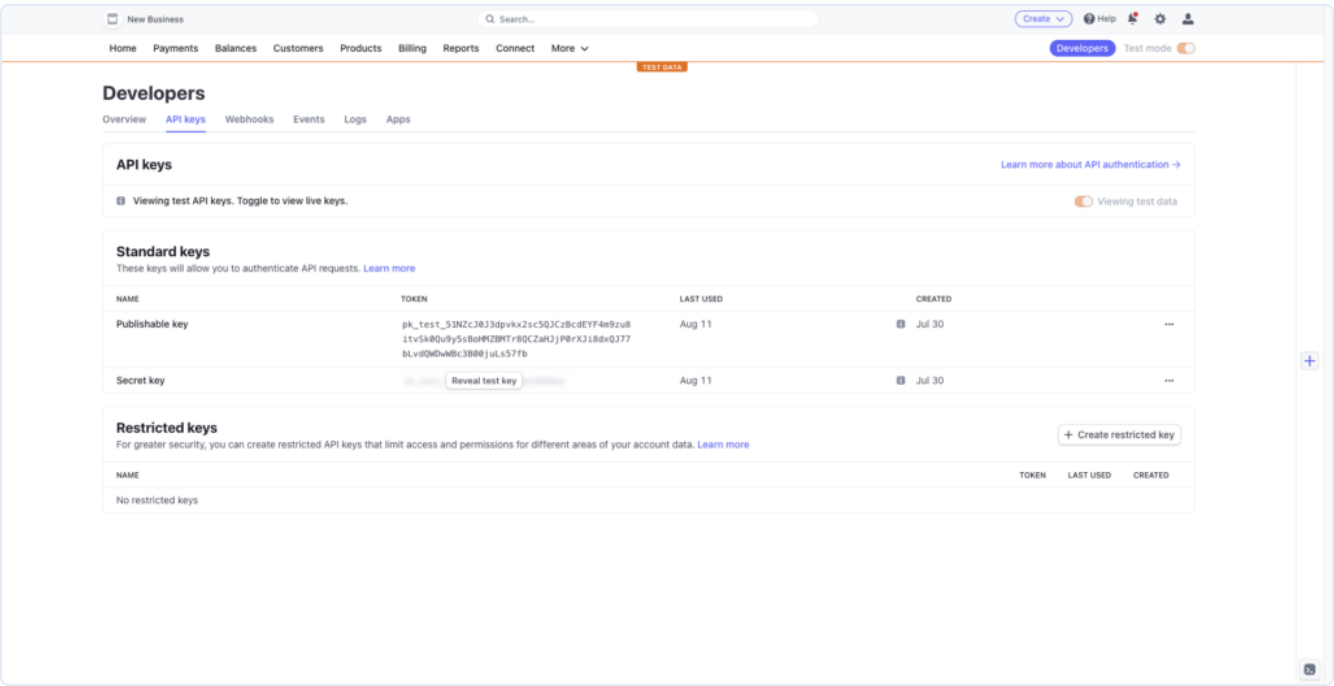
After payment, Stripe redirects back to your app, enabling post-payment tasks such as confirmations and delivery requests. For reliability, configure a backend webhook to update Stripe, ensuring payment data retention even if customers accidentally close the page after

payment.

While effective, this method lacks flexibility in customization and design. It can also be tricky to set up correctly for mobile apps, where a native integration would look far more seamless.

API Keys

When working with the Stripe API, you will need access to API keys for your client and server apps to interact with the Stripe backend. You can access your Stripe API keys on [your Stripe developer dashboard](#). Here's what it would look like:



— The Stripe dashboard showing API keys

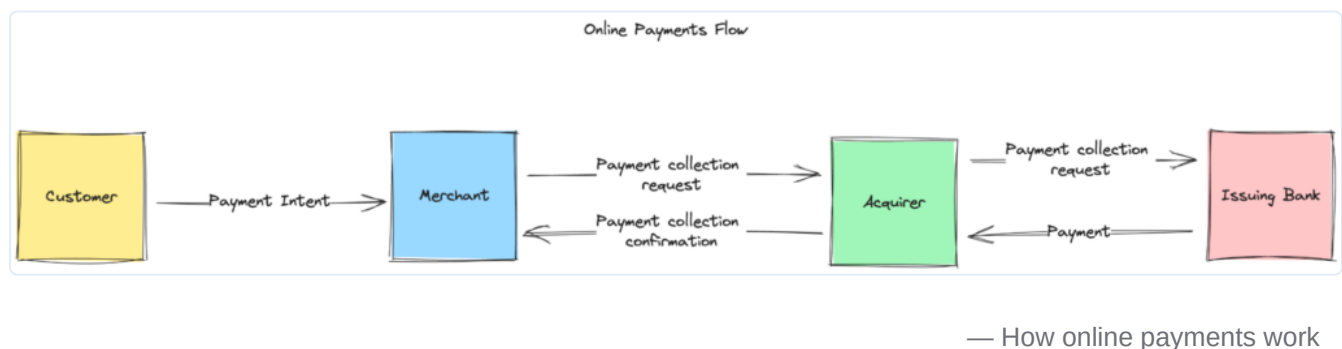
How Do Payments in Stripe Work?

To understand how payments in Stripe work, you need to understand all the stakeholders involved. Four stakeholders are involved in each payment transaction:

1. **Customer:** The person who intends to pay for a service/product.

2. **Merchant:** You, the business owner, are responsible for receiving payments and selling services/products.
3. **Acquirer:** A bank that processes payments on behalf of you (the merchant) and routes your payment request to your customer's banks. Acquirers may partner with a third party to help process payments.
4. **Issuing bank:** The bank that extends credit and issues cards and other payment methods to consumers.

Here's a typical payment flow between these stakeholders at a very high level.



The customer lets the merchant know they're willing to pay. The merchant then forwards the payment-related details to their acquiring bank, which collects the payment from the customer's issuing bank and lets the merchant know the payment was successful.

This is a very high-level overview of the payment process. As a merchant, you only need to worry about collecting the payment intent, passing it on to the payment processor, and handling the payment result. However, as discussed earlier, there are two ways you could go about it.

When creating a Stripe-managed checkout session where Stripe takes care of your payment details collection, here's what the typical flow looks like:



— The Stripe hosted checkout payment workflow. (Source: [Stripe Docs](#))

With custom payment flows, it's really up to you. You can design the interaction between your client, server, customer, and the Stripe API based on your app's needs. You can add address collection, invoice generation, cancellation, free trials, etc., to this workflow as you need.

Now that you understand how Stripe payments work, you are ready to start building it in your Java application.

Stripe Integration in Spring Boot Application

To begin the Stripe integration, create a frontend app to interact with the Java backend and initiate payments. In this tutorial, you'll build a React app to trigger various payment types and subscriptions so you gain a clear understanding of their mechanisms.

Note: This tutorial won't cover building a complete ecommerce site; it's primarily aimed at guiding you through the straightforward process of integrating Stripe into Spring Boot.

Setting Up the Frontend and Backend Projects

Create a new directory and scaffold a React project using Vite by running the following command:

```
npm create vite@latest
```

Set the project name as **frontend** (or any preferred name), framework as **React** and variant as **TypeScript**. Navigate to the project directory and install Chakra UI for quick scaffolding of UI elements by running the following command:

```
npm i @chakra-ui/react @emotion/react @emotion/styled framer-motion
```

You will also install `react-router-dom` in your project for client-side routing by running the command below:

```
npm i react-router-dom
```

Now, you are ready to start building your frontend app. Here's the homepage you are going to build.

Stripe Payments With React & Java

[Integrated Checkout](#)[Hosted Checkout](#)[New Subscription](#)[Cancel Subscription](#)[Subscription With Trial](#)[View Invoices](#)

— The completed home page for the frontend app.

Clicking any button on this page will take you to separate checkout pages with payment forms. To begin, create a new folder named **routes** in your **frontend/src** directory. Inside this folder, create a **Home.tsx** file. This file will hold the code for your app's home route (/). Paste the following code into the file:


```

import {Button, Center, Heading, VStack} from "@chakra-ui/react";

import { useNavigate } from "react-router-dom";

function Home() {
  const navigate = useNavigate()
  const navigateToIntegratedCheckout = () => {
    navigate("/integrated-checkout")
  }

  const navigateToHostedCheckout = () => {
    navigate("/hosted-checkout")
  }

  const navigateToNewSubscription = () => {
    navigate("/new-subscription")
  }

  const navigateToCancelSubscription = () => {
    navigate("/cancel-subscription")
  }

  const navigateToSubscriptionWithTrial = () => {
    navigate("/subscription-with-trial")
  }

  const navigateToViewInvoices = () => {
    navigate("/view-invoices")
  }

  return (
    <>
      <Center h={'100vh'} color='black'>
        <VStack spacing='24px'>
          <Heading>Stripe Payments With React & Java</Heading>
          <Button
            colorScheme={'teal'}

```

```

        onClick={navigateToIntegratedCheckout}>
        Integrated Checkout
      </Button>
      <Button
        colorScheme={'blue'}
        onClick={navigateToHostedCheckout}>
        Hosted Checkout
      </Button>
      <Button
        colorScheme={'yellow'}
        onClick={navigateToNewSubscription}>
        New Subscription
      </Button>
      <Button
        colorScheme={'purple'}
        onClick={navigateToCancelSubscription}>
        Cancel Subscription
      </Button>
      <Button
        colorScheme={'facebook'}
        onClick={navigateToSubscriptionWithTrial}>
        Subscription With Trial
      </Button>
      <Button
        colorScheme={'pink'}
        onClick={navigateToViewInvoices}>
        View Invoices
      </Button>
    </VStack>
  </Center>
</>
)
}

export default Home

```

To enable navigation in your app, update your **App.tsx** file to configure the `RouteProvider` class from `react-router-dom`.

```

import Home from "../routes/Home.tsx";
import {
  createBrowserRouter,
  RouterProvider,
} from "react-router-dom";

function App() {

  const router = createBrowserRouter([
    {
      path: "/",
      element: (
        <Home/>
      ),
    },
  ]);

  return (
    <RouterProvider router={router}/>
  )
}

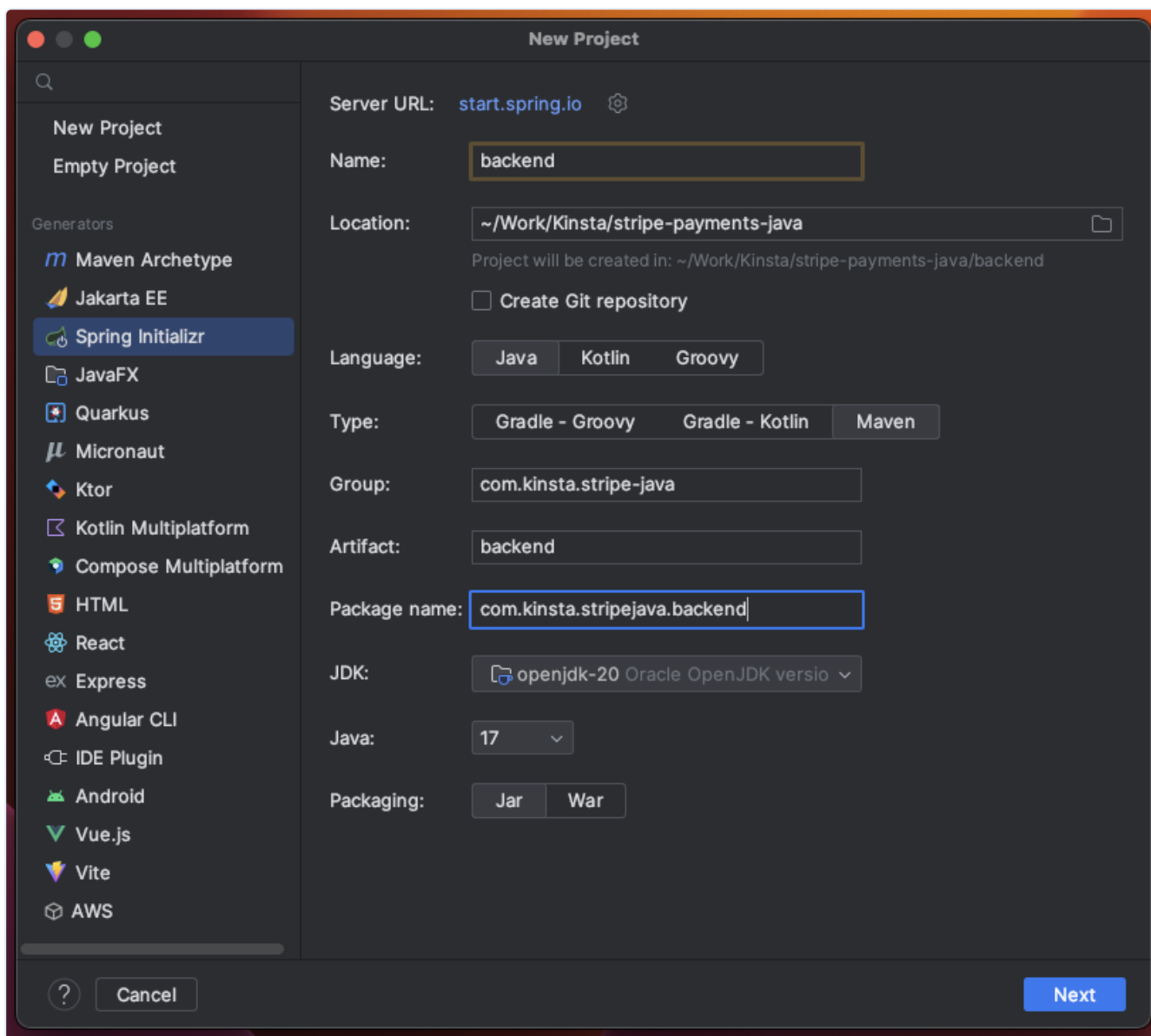
export default App

```

Run the `npm run dev` command to preview your application on <https://localhost:5173>.

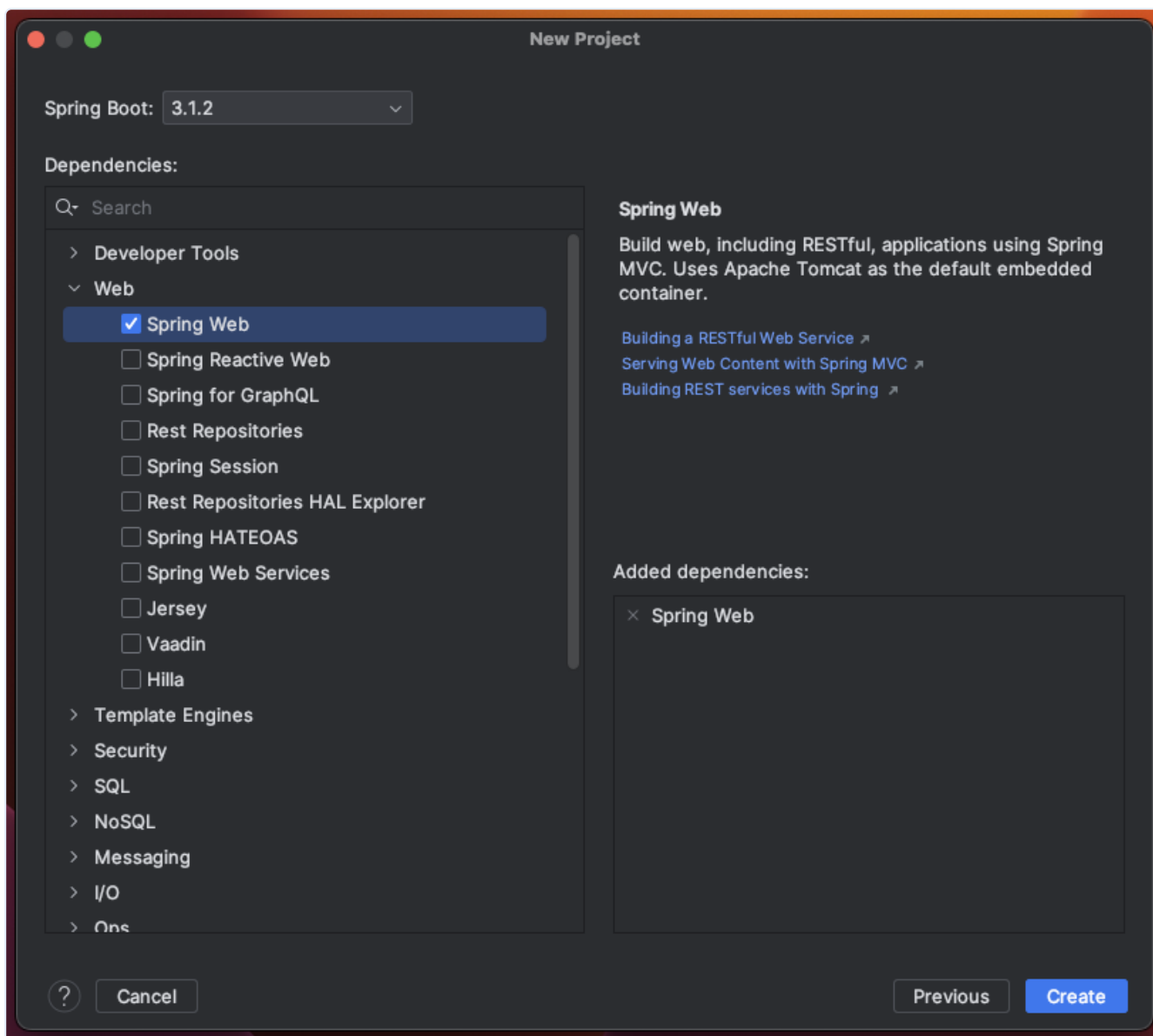
This completes the initial setup needed for the frontend app. Next, create a backend app using Spring Boot. To initialize the app, you can use the [spring initializr](#) website (If your IDE supports creating Spring apps, you don't need to use the website).

IntelliJ IDEA supports creating Spring Boot apps. Start by choosing the **New Project** option on IntelliJ IDEA. Then, choose **Spring Initializr** from the left pane. Input your backend project details: name (**backend**), location (**stripe-payments-java** directory), language (**Java**), and type (**Maven**). For group and artifact names, use **com.kinsta.stripe-java** and **backend**, respectively.



— The IDEA new project dialog.

Click **Next** button. Then, add dependencies to your project by choosing **Spring Web** from the **Web** dropdown in the dependencies pane and click the **Create** button.



— Choosing the dependencies.

This will create the Java project and open it in your IDE. You can now proceed with creating the various checkout flows using Stripe.

Accepting Online Payments for Product Purchases

The most important and widely used functionality of Stripe is to accept one-off payments from customers. In this section, you will learn two ways to integrate payment processing in your app with Stripe.

Hosted Checkout

First, you build a checkout page that triggers a hosted payment workflow where you only trigger a payment from your frontend app. Then Stripe takes care of collecting the customer's card details and collecting the payment and only shares the result of the payment operation at the end.

This is what the checkout page would look like:

Hosted Checkout Example



Puma Shoes
Premium Shoes
Quantity: 1
\$20



Nike Sliders
Comfortable everyday slippers
Quantity: 1
\$10

Total **\$30**

— The completed hosted checkout page.

This page has three main components: `CartItem` — represents each cart item; `TotalFooter` — displays the total amount; `CustomerDetails` — collects customer details. You to reuse these components to build checkout forms for other scenarios in this article, such as integrated checkout and subscriptions.

Building the Frontend

Create a **components** folder in your **frontend/src** directory. In the **components** folder, create a new file **CartItem.tsx** and paste the following code:

```
import {Button, Card, CardBody, CardFooter, Heading, Image, Stack, Text} from '@sveltejs/svelte-materials';

function CartItem(props: CartItemProps) {
  return <Card direction={{base: 'column', sm: 'row'}}
    overflow='hidden'
    width={'xl'}
    variant='outline'>
    <Image
      objectFit='cover'
      maxW={{base: '100%', sm: '200px'}}
      src={props.data.image}
    />
    <Stack mt='6' spacing='3'>
      <CardBody>
        <VStack spacing={'3'} alignItems={"flex-start"}>
          <Heading size='md'>{props.data.name}</Heading>
          <VStack spacing={'1'} alignItems={"flex-start"}>
            <Text>
              {props.data.description}
            </Text>
            {(props.mode === "checkout" ? <Text>
              {"Quantity: " + props.data.quantity}
            </Text> : <></>)}
          </VStack>
        </CardBody>
      </Stack>
    </Card>
  
```

```

        </VStack>
      </CardBody>

      <CardFooter>
        <VStack alignItems={'flex-start'}>
          <Text color='blue.600' fontSize='2xl'>
            {"$" + props.data.price}
          </Text>
        </VStack>
      </CardFooter>
    </Stack>
  </Card>
}

export interface ItemData {
  name: string
  price: number
  quantity: number
  image: string
  description: string
  id: string
}

interface CartItemProps {
  data: ItemData
  mode: "subscription" | "checkout"
  onCancelled?: () => void
}

export default CartItem

```

The code above defines two interfaces to use as types for the properties passed to the component. The `ItemData` type is exported for reuse in other components.

The code returns a cart item component's layout. It utilizes the provided props to render the item on the screen.

Next, create a **TotalFooter.tsx** file in the **components** directory and paste the following code:

```
import {Divider, HStack, Text} from "@chakra-ui/react";

function TotalFooter(props: TotalFooterProps) {
  return <>
    <Divider />
    <HStack>
      <Text>Total</Text>
      <Text color='blue.600' fontSize='2xl'>
        {"$" + props.total}
      </Text>
    </HStack>
    {props.mode === "subscription" &&
      <Text fontSize={"xs"}>(Monthly, starting today)</Text>
    }
    {props.mode === "trial" &&
      <Text fontSize={"xs"}>(Monthly, starting next month)</Text>
    }
  </>
}

interface TotalFooterProps {
  total: number
  mode: "checkout" | "subscription" | "trial"
}

export default TotalFooter
```

The TotalFooter component displays the total value for the cart and uses the mode value to [conditionally render](#) specific text.

Finally, create the CustomerDetails.tsx component and paste the following code:

```

import {ItemData} from "../CartItem.tsx";
import {Button, Input, VStack} from "@chakra-ui/react";
import {useState} from "react";

function CustomerDetails(props: CustomerDetailsProp) {
  const [name, setName] = useState("")
  const [email, setEmail] = useState("")
  const onCustomerNameChange = (ev: React.ChangeEvent<HTMLInput
    setName(ev.target.value)
  }

  const onCustomerEmailChange = (ev: React.ChangeEvent<HTMLInput
    setEmail(ev.target.value)
  }

  const initiatePayment = () => {
    fetch(process.env.VITE_SERVER_BASE_URL + props.endpoint, {
      method: "POST",
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        items: props.data.map(elem => ({name: elem.name,
        customerName: name,
        customerEmail: email,
      })
    })
    .then(r => r.text())
    .then(r => {
      window.location.href = r
    })
  })

  return <>
    <VStack spacing={3} width={'xl'}>
      <Input variant='filled' placeholder='Customer Name' on

```

```

        <Input variant='filled' placeholder='Customer Email' />
        <Button onClick={initiatePayment} colorScheme='green' />
      </VStack>
    </>
  }

  interface CustomerDetailsProp {
    data: ItemData[]
    endpoint: string
  }

  export default CustomerDetails

```

The code above displays a form with two input fields — to collect the user’s name and email. When the **Checkout** button is clicked, the `initiatePayment` method is invoked to send the checkout request to the backend.

It requests the endpoint that you’ve passed to the component and sends the customer’s information and cart items as part of the request, then redirects the user to the URL received from the server. This URL will lead the user to a checkout page hosted on Stripe’s server. You will get to building this URL in a bit.

Note: This component uses the environment variable `VITE_SERVER_BASE_URL` for the backend server URL. Set it by creating `.env` file in the root of your project:

```
VITE_SERVER_BASE_URL=http://localhost:8080
```

All components have been created. Now, let’s proceed to build the hosted checkout route using the components. To do that, create a new **HostedCheckout.tsx** file in your **routes** folder with the following code:

```

import {Center, Heading, VStack} from "@chakra-ui/react";
import {useState} from "react";
import CartItem, {ItemData} from "../components/CartItem.tsx";
import TotalFooter from "../components/TotalFooter.tsx";
import CustomerDetails from "../components/CustomerDetails.tsx";
import {Products} from '../data.ts'

function HostedCheckout() {
  const [items] = useState<ItemData[]>(Products)
  return <>
    <Center h={'100vh'} color='black'>
      <VStack spacing='24px'>
        <Heading>Hosted Checkout Example</Heading>
        {items.map(elem => {
          return <CartItem data={elem} mode={'checkout'} />
        })}
        <TotalFooter total={30} mode={"checkout"} />
        <CustomerDetails data={items} endpoint={"/checkout/hosted"} />
      </VStack>
    </Center>
  </>
}

export default HostedCheckout

```

This route utilizes the three components you've just constructed to assemble a checkout screen. All component modes are configured as **checkout**, and the endpoint `/checkout/hosted` is provided to the form component for initiating the checkout request accurately.

The component uses a `Products` object to fill the items array. In real-world scenarios, this data comes from your cart API, containing the user's selected items. However, for this tutorial, a static list from a script populates the array. Define the array by creating a **data.ts** file at your frontend project's root and storing the following code within it:

```
import {ItemData} from "../components/CartItem.tsx";

export const Products: ItemData[] = [
  {
    description: "Premium Shoes",
    image: "https://source.unsplash.com/NUoPWImmjCU",
    name: "Puma Shoes",
    price: 20,
    quantity: 1,
    id: "shoe"
  },
  {
    description: "Comfortable everyday slippers",
    image: "https://source.unsplash.com/K_gIPI791Jo",
    name: "Nike Sliders",
    price: 10,
    quantity: 1,
    id: "slippers"
  },
]
```

This file defines two items in the products array that are rendered in the cart. Feel free to tweak the values of the products.

As the last step of building the frontend, create two new routes to handle success and failure. The Stripe-hosted checkout page would redirect users into your app on these two routes based on the transaction result. Stripe will also provide your routes with transaction-related payload, such as the checkout session ID, which you can use to [retrieve the corresponding checkout session object](#) and access checkout related data like the payment method, invoice details, etc.

To do that, create a **Success.tsx** file in the **src/routes** directory and save the following code in it:

```

import {Button, Center, Heading, Text, VStack} from "@chakra-ui/react"
import {useNavigate} from "react-router-dom";

function Success() {
  const queryParams = new URLSearchParams(window.location.search)
  const navigate = useNavigate()
  const onClick = () => {
    navigate("/")
  }
  return <Center h={'100vh'} color='green'>
    <VStack spacing={3}>
      <Heading fontSize={'4xl'}>Success!</Heading>
      <Text color={'black'}>{queryParams.toString().split("&")}
      <Button onClick={onClick} colorScheme={'green'}>Go Home</Button>
    </VStack>
  </Center>
}

export default Success

```

Upon rendering, this component shows the “Success!” message and prints any URL query parameters on the screen. It also includes a button to redirect users to the application’s homepage.

When building real-world apps, this page is where you would handle non-critical app-side transactions that depend on the success of the transaction at hand. For instance, if you are building a checkout page for an online store, you could use this page to show a confirmation to the user and an ETA on when their purchased products would be delivered.

Next, create a **Failure.tsx** file with the following code in it:

```

import {Button, Center, Heading, Text, VStack} from "@chakra-ui/react";
import {useNavigate} from "react-router-dom";

function Failure() {
  const queryParams = new URLSearchParams(window.location.search);
  const navigate = useNavigate();
  const onClick = () => {
    navigate("/")
  }

  return <Center h={'100vh'} color='red'>
    <VStack spacing={3}>
      <Heading fontSize={'4xl'}>Failure!</Heading>
      <Text color={'black'}>{queryParams.toString().split("&")[0]}</Text>
      <Button onClick={onClick} colorScheme='red'>Try Again</Button>
    </VStack>
  </Center>
}

export default Failure

```

This component is similar to that of **Success.tsx** and displays the “Failure!” message when rendered.

Info

Avoid passing critical information or carrying out critical operations on either the success or failure page, as the customer may never make it to either page if they close the browser tab or lose connectivity.

For essential tasks such as product delivery, sending emails, or any critical part of your purchase flow, use [webhooks](#). Webhooks are API routes on your server that Stripe can invoke when a transaction occurs.

The webhook receives full transaction details (via the `CheckoutSession` object), allowing you to log it into your app database and trigger corresponding success or failure workflows. As your server is always accessible to Stripe, no transactions are missed, ensuring your online store's consistent functionality.

Finally, update the **App.tsx** file to make it look like this:

```
import Home from "../routes/Home.tsx";
import {createBrowserRouter, RouterProvider,} from "react-router-dom";
import HostedCheckout from "../routes/HostedCheckout.tsx";
import Success from "../routes/Success.tsx";
import Failure from "../routes/Failure.tsx";

function App() {

  const router = createBrowserRouter([
    {
      path: "/",
      element: (
        <Home/>
      ),
    },
    {
      path: "/hosted-checkout",
      element: (
        <HostedCheckout/>
      )
    },
    {
      path: '/success',
      element: (
```



```

        <Success/>
    )
},
{
    path: '/failure',
    element: (
        <Failure/>
    )
},
]);

return (
    <RouterProvider router={router}/>
)
}

export default App

```

This will make sure that the `Success` and `Failure` components are rendered on the `/success` and `/failure` routes, respectively.

This completes the frontend setup. Next, set up the backend to create the `/checkout/hosted` endpoint.

Building the Backend

Open the backend project and install the Stripe SDK by adding the following lines in the dependencies array in your **pom.xml** file:

```

<dependency>
  <groupId>com.stripe</groupId>
  <artifactId>stripe-java</artifactId>

```

```
<version>22.29.0</version>
</dependency>
```

Next, [load Maven changes](#) in your project to install dependencies. If your IDE doesn't support this through the UI, execute either the `maven dependency:resolve` or `maven install` command. If you don't have the maven CLI, use the mvnw wrapper from Spring initializr when you create the project.

Once dependencies are installed, create a new REST controller to handle incoming HTTP requests for your backend app. To do this, create a **PaymentController.java** file in the **src/main/java/com/kinsta/stripe-java/backend** directory and add the following code:

```
package com.kinsta.stripejava.backend;

import com.stripe.Stripe;
import com.stripe.exception.StripeException;
import com.stripe.model.Customer;
import com.stripe.model.Product;
import com.stripe.model.checkout.Session;
import com.stripe.param.checkout.SessionCreateParams;
import com.stripe.param.checkout.SessionCreateParams.LineItem.PriceData;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RestController;

@RestController
@CrossOrigin
public class PaymentController {

    String STRIPE_API_KEY = System.getenv().get("STRIPE_API_KEY");
```

```
        @PostMapping("/checkout/hosted")
        String hostedCheckout(@RequestBody RequestDTO requestDTO) throws IOException {
            return "Hello World!";
        }
    }
}
```

The code above imports essential Stripe dependencies and establishes the `PaymentController` class. This class carries two annotations: `@RestController` and `@CrossOrigin`. The `@RestController` annotation instructs Spring Boot to treat this class as a controller, and its methods can now use `@Mapping` annotations to handle incoming HTTP requests.

The `@CrossOrigin` annotation marks all endpoints defined in this class as open to all origins under the CORS rules. However, this practice is discouraged in production due to potential security vulnerabilities from various internet domains.

For optimal results, it's advisable to host both backend and frontend servers on the same domain to circumvent CORS problems. Alternatively, if this isn't feasible, you can specify the domain of your frontend client (which sends requests to the backend server) using the `@CrossOrigin` annotation, like this:

```
@CrossOrigin(origins = "http://frontend.com")
```

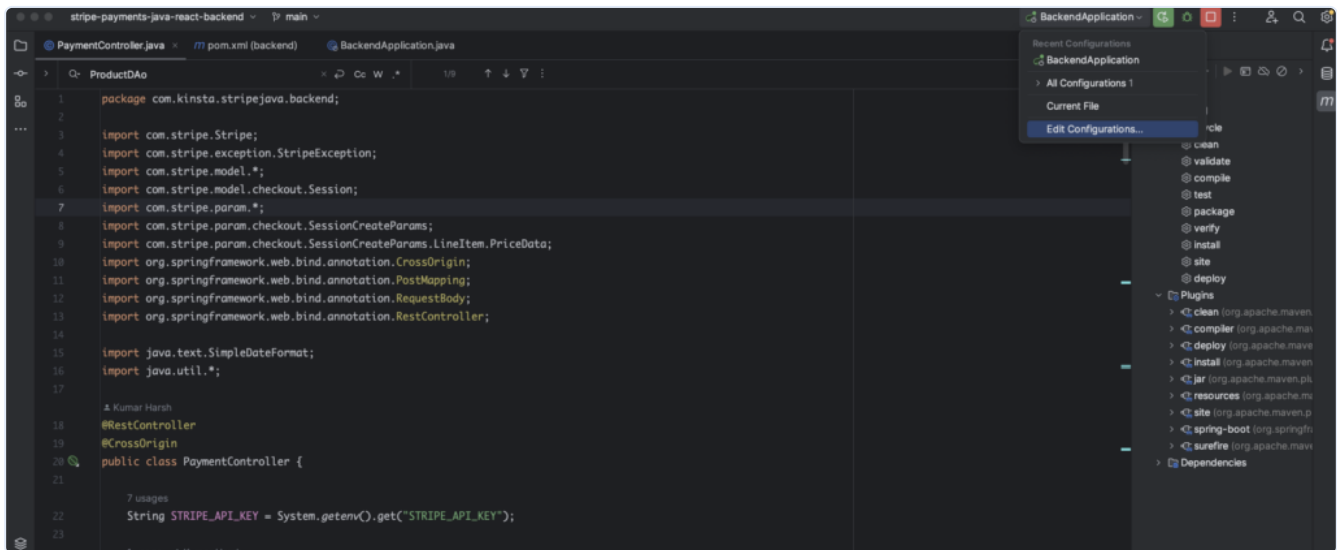
The `PaymentController` class will extract the Stripe API key from environment variables to provide to the Stripe SDK later. When running the application, you must provide your Stripe API key to the app through environment variables.

Locally, you can create a new environment variable in your system either temporarily (by adding a `KEY=VALUE` phrase before the command used to start your development server) or permanently (by updating your terminal's config files or setting an environment variable in the

control panel in Windows).

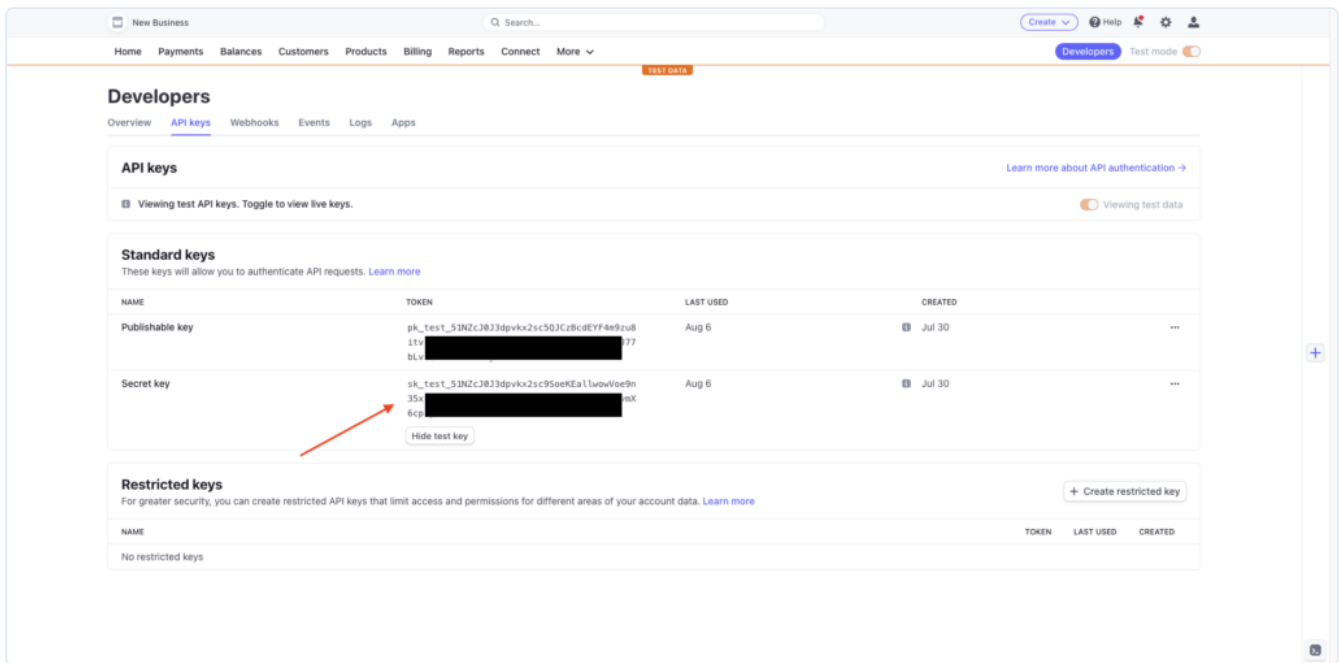
In production environments, your deployment provider (such as Kinsta) will provide you with a separate option to fill in the environment variables used by your application.

If you are using IntelliJ IDEA (or a similar IDE), click **Run Configurations** at the top right of the IDE and click **Edit Configurations...** option from the dropdown list that opens to update your run command and set the environment variable.



— Opening the run/debug configurations dialog box.

This will open up a dialog where you can provide the environment variables for your app using the **Environment variables** field. Enter the environment variable `STRIPE_API_KEY` in the format `VAR1=VALUE`. You can find your API key on the [Stripe Developers website](#). You must provide the value of the **Secret Key** from this page.



— The Stripe dashboard showing API keys.

If you haven't already, [create a new Stripe account](#) to get access to the API keys.

Once you have set up the API key, proceed to build the endpoint. This endpoint will collect the customer data (name and email), create a customer profile for them in Stripe if one doesn't exist already, and create a [Checkout Session](#) to allow users to pay for the cart items.

Here's what the code for the hostedCheckout method looks like:

```
@PostMapping("/checkout/hosted")
String hostedCheckout(@RequestBody RequestDTO requestDTO) throws IOException {
    Stripe.apiKey = STRIPE_API_KEY;
    String clientBaseUrl = System.getenv().get("CLIENT_BASE_URL");

    // Start by finding an existing customer record from Stripe
```

```

        Customer customer = CustomerUtil.findOrCreateCustomer(request);

        // Next, create a checkout session by adding the details of the items
        SessionCreateParams.Builder paramsBuilder =
            SessionCreateParams.builder()
                .setMode(SessionCreateParams.Mode.PAYMENT)
                .setCustomer(customer.getId())
                .setSuccessUrl(clientBaseUrl + "/success?session=")
                .setCancelUrl(clientBaseUrl + "/failure");

        for (Product product : requestDTO.getItems()) {
            paramsBuilder.addLineItem(
                SessionCreateParams.LineItem.builder()
                    .setQuantity(1L)
                    .setPriceData(
                        PriceData.builder()
                            .setProductData(
                                PriceData.ProductData.builder()
                                    .putMetadata("name", product.getName())
                                    .setName(product.getName())
                                    .build()
                                )
                            .setCurrency(ProductData.getCurrency())
                            .setUnitAmountDecimal(product.getPrice())
                            .build()
                        )
                    .build());
        }

        Session session = Session.create(paramsBuilder.build());

        return session.getUrl();
    }
}

```

When building the checkout session, the code uses the name of the product received from the client but does not use the price details from the request. This approach avoids potential client-side price manipulation, where malicious actors might send reduced prices in the

checkout request to pay less for products and services.

To prevent this, the `hostedCheckout` method queries your products database (via `ProductDAO`) to retrieve the correct item price.

Additionally, Stripe offers various `Builder` classes following the builder design pattern. These classes aid in creating parameter objects for Stripe requests. The provided code snippet also references environment variables to fetch the client app's URL. This URL is necessary for the checkout session object to redirect appropriately after successful or failed payments.

To execute this code, set the client app's URL via environment variables, similar to how the Stripe API key was provided. As the client app runs through Vite, the local app URL should be `http://localhost:5173`. Include this in your environment variables through your IDE, terminal, or system control panel.

```
CLIENT_BASE_URL=http://localhost:5173
```

Also, provide the app with a `ProductDAO` to look up product prices from. Data Access Object (DAO) interacts with data sources (such as databases) to access app-related data. While setting up a products database would be outside the scope of this tutorial, a simple implementation you can do would be to add a new file **`ProductDAO.java`** in the same directory as the **`PaymentController.java`** and paste the following code:

```
package com.kinsta.stripejava.backend;

import com.stripe.model.Price;
import com.stripe.model.Product;
```

```
import java.math.BigDecimal;

public class ProductDAO {

    static Product[] products;

    static {
        products = new Product[4];

        Product sampleProduct = new Product();
        Price samplePrice = new Price();

        sampleProduct.setName("Puma Shoes");
        sampleProduct.setId("shoe");
        samplePrice.setCurrency("usd");
        samplePrice.setUnitAmountDecimal(BigDecimal.valueOf(2000))
        sampleProduct.setDefaultPriceObject(samplePrice);
        products[0] = sampleProduct;

        sampleProduct = new Product();
        samplePrice = new Price();

        sampleProduct.setName("Nike Sliders");
        sampleProduct.setId("slippers");
        samplePrice.setCurrency("usd");
        samplePrice.setUnitAmountDecimal(BigDecimal.valueOf(1000))
        sampleProduct.setDefaultPriceObject(samplePrice);
        products[1] = sampleProduct;

        sampleProduct = new Product();
        samplePrice = new Price();

        sampleProduct.setName("Apple Music+");
        sampleProduct.setId("music");
        samplePrice.setCurrency("usd");
        samplePrice.setUnitAmountDecimal(BigDecimal.valueOf(499))
        sampleProduct.setDefaultPriceObject(samplePrice);
        products[2] = sampleProduct;
```



```

    }

    public static Product getProduct(String id) {

        if ("shoe".equals(id)) {
            return products[0];
        } else if ("slippers".equals(id)) {
            return products[1];
        } else if ("music".equals(id)) {
            return products[2];
        } else return new Product();
    }
}

```

This will initialize an array of products and allow you to query product data using its identifier (ID). You will also need to create a [DTO](#) (Data Transfer Object) to allow Spring Boot to automatically serialize the incoming payload from the client and present you with a simple object to access the data. To do that, create a new file **RequestDTO.java** and paste the following code:

```

package com.kinsta.stripejava.backend;

import com.stripe.model.Product;

public class RequestDTO {
    Product[] items;
    String customerName;
    String customerEmail;

    public Product[] getItems() {
        return items;
    }
}

```

```

    }

    public String getCustomerName() {
        return customerName;
    }

    public String getCustomerEmail() {
        return customerEmail;
    }
}

```

This file defines a [POJO](#) that carries the customer name, email, and the list of items they are checking out with.

Finally, implement the `CustomerUtil.findOrCreateCustomer()` method to create the Customer object in Stripe if it does not exist already. To do that, create a file with the name `CustomerUtil` and add the following code to it:

```

package com.kinsta.stripejava.backend;

import com.stripe.exception.StripeException;
import com.stripe.model.Customer;
import com.stripe.model.CustomerSearchResult;
import com.stripe.param.CustomerCreateParams;
import com.stripe.param.CustomerSearchParams;

public class CustomerUtil {

    public static Customer findCustomerByEmail(String email) throws
        StripeException {
        CustomerSearchParams params =
            CustomerSearchParams
                .builder()

```

```

        .setQuery("email:" + email + "")
        .build();

    CustomerSearchResult result = Customer.search(params);

    return result.getData().size() > 0 ? result.getData().get(0) : null;
}

public static Customer findOrCreateCustomer(String email, String name) {
    CustomerSearchParams params =
        CustomerSearchParams
            .builder()
            .setQuery("email:" + email + "")
            .build();

    CustomerSearchResult result = Customer.search(params);

    Customer customer;

    // If no existing customer was found, create a new record
    if (result.getData().size() == 0) {
        CustomerCreateParams customerCreateParams = CustomerCreateParams
            .builder()
            .setName(name)
            .setEmail(email)
            .build();

        customer = Customer.create(customerCreateParams);
    } else {
        customer = result.getData().get(0);
    }

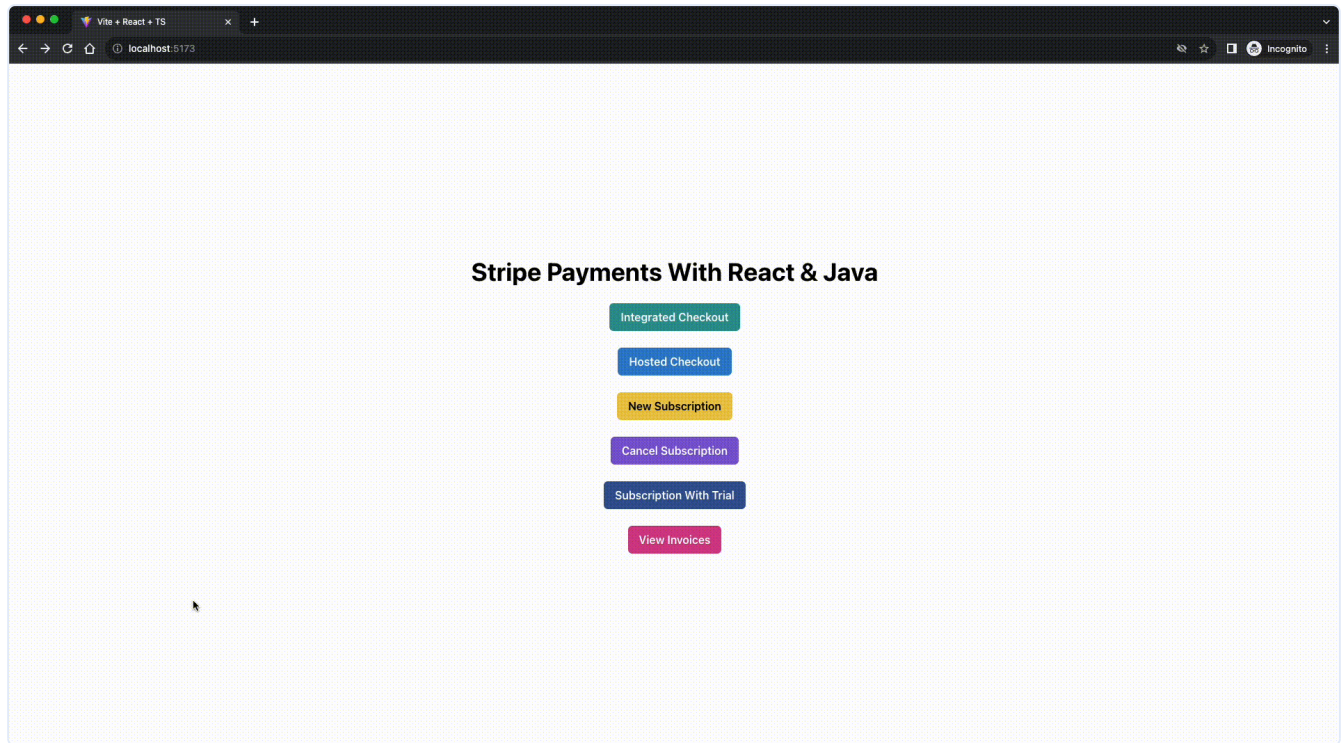
    return customer;
}
}

```

This class also contains another method `findCustomerByEmail` that allows you to look up customers in Stripe using their email addresses. The [Customer Search API](#) is used to look up

the customer records in the Stripe database and [Customer Create API](#) is used to create the customer records as needed.

This completes the backend setup needed for the hosted checkout flow. You can now test the app by running the frontend and the backend apps in their IDEs or separate terminals. Here's what the success flow would look like:



— A successful hosted checkout flow.

When testing Stripe integrations, you can always use the following card details to simulate card transactions:

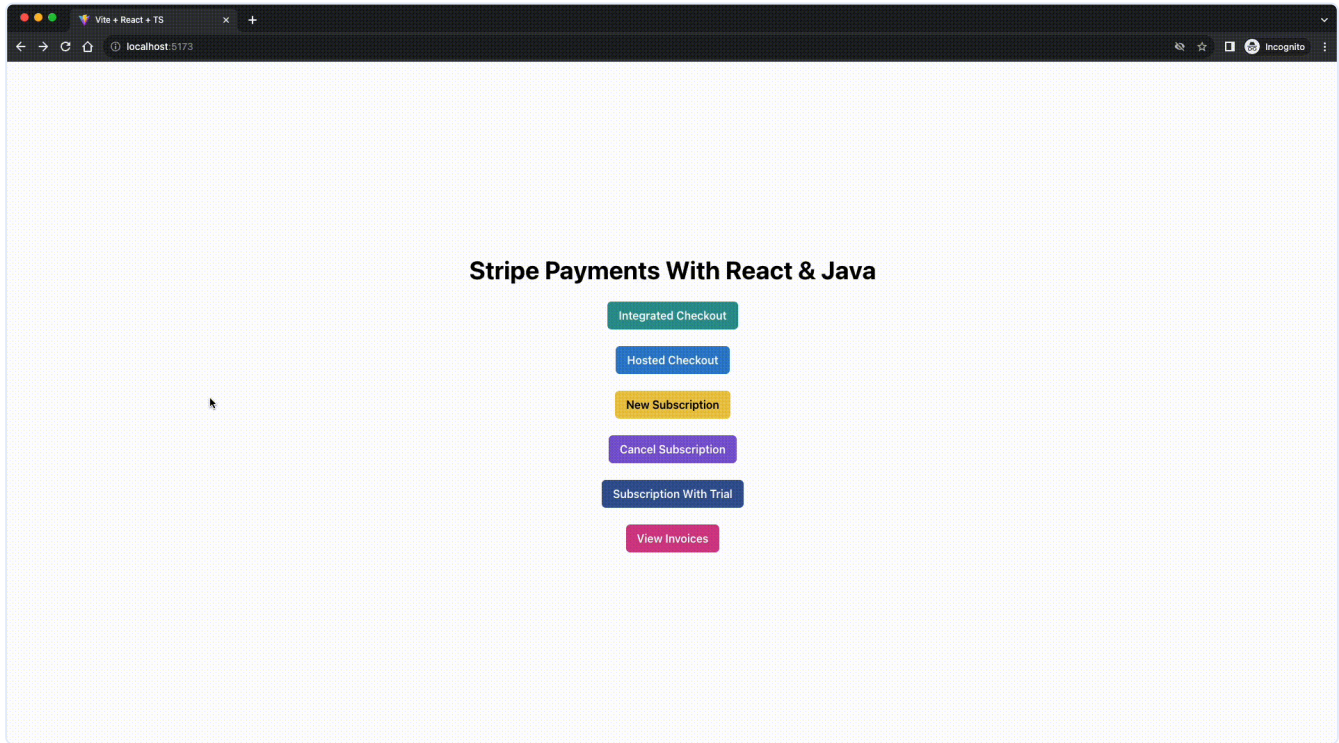
Card Number: 4111 1111 1111 1111

Expiry Month & Year: 12 / 25

CVV: Any three-digit number

Name on Card: Any Name

If you choose to cancel the transaction instead of paying, here's what the failure flow would look like:



— A failed hosted checkout flow.

This completes the setup of a Stripe-hosted checkout experience built into your app. You can look through the [Stripe docs](#) to learn more about how to customize your checkout page, collect more details from the customer, and more.

Integrated Checkout

An integrated checkout experience refers to building a payment flow that does not redirect your users outside your application (like it did in the hosted checkout flow) and renders the payment form in your app itself.

Building an integrated checkout experience means handling customers' payment details, which entails sensitive information such as credit card numbers, Google Pay ID, etc. Not all

apps are designed to handle this data securely.

To remove the burden of meeting standards like PCI-DSS, Stripe provides [elements](#) that you can use in-app to collect payment details while still letting Stripe manage the security and process the payments securely on their end.

Building the Frontend

To start, install the [Stripe React SDK](#) in your frontend app to access the Stripe Elements by running the following command in your frontend directory:

```
npm i @stripe/react-stripe-js @stripe/stripe-js
```

Next, create a new file called **IntegratedCheckout.tsx** in your **frontend/src/routes** directory and save the following code in it:

```
import {Button, Center, Heading, Input, VStack} from "@chakra-ui/react";
import {useEffect, useState} from "react";
import CartItem, {ItemData} from "../components/CartItem.tsx";
import TotalFooter from "../components/TotalFooter.tsx";
import {Products} from '../data.ts'
import {Elements, PaymentElement, useElements, useStripe} from '@stripe/react-stripe-js';
import {loadStripe, Stripe} from '@stripe/stripe-js';

function IntegratedCheckout() {

  const [items] = useState<ItemData[]>(Products)
  const [transactionClientSecret, setTransactionClientSecret] =
  const [stripePromise, setStripePromise] = useState<Promise<Stripe>>>
```

```

const [name, setName] = useState("")
const [email, setEmail] = useState("")
const onCustomerNameChange = (ev: React.ChangeEvent<HTMLInput
    setName(ev.target.value)
}

const onCustomerEmailChange = (ev: React.ChangeEvent<HTMLInput
    setEmail(ev.target.value)
}

useEffect(() => {
    // Make sure to call `loadStripe` outside of a component's
    // recreating the `Stripe` object on every render.
    setStripePromise(loadStripe(process.env.VITE_STRIPE_API_KI

}, []))

const createTransactionSecret = () => {
    fetch(process.env.VITE_SERVER_BASE_URL + "/checkout/integr
        method: "POST",
        headers: {'Content-Type': 'application/json'},
        body: JSON.stringify({
            items: items.map(elem => ({name: elem.name, id: e
            customerName: name,
            customerEmail: email,
        }))
    })
    .then(r => r.text())
    .then(r => {
        setTransactionClientSecret(r)
    })
}

return <>
    <Center h={'100vh'} color='black'>
        <VStack spacing='24px'>
            <Heading>Integrated Checkout Example</Heading>
            {items.map(elem => {
                return <CartItem data={elem} mode={'checkout'}

```

```

    )))
    <TotalFooter total={30} mode={"checkout"}/>

    <Input variant='filled' placeholder='Customer Name'
          value={name}/>
    <Input variant='filled' placeholder='Customer Email'
          value={email}/>
    <Button onClick={createTransactionSecret} colorScheme="dark">
      {(transactionClientSecret === "" ?
        <></>
        : <Elements stripe={stripePromise} options={{stripePublicKey}}>
          <CheckoutForm/>
        </Elements>)}
    </VStack>
  </Center>
</>
}

```

```

const CheckoutForm = () => {

  const stripe = useStripe();
  const elements = useElements();
  const handleSubmit = async (event: React.MouseEvent<HTMLButtonElement>) => {
    event.preventDefault();

    if (!stripe || !elements) {
      return;
    }

    const result = await stripe.confirmPayment({
      elements,
      confirmParams: {
        return_url: process.env.VITE_CLIENT_BASE_URL + "/",
      },
    });

    if (result.error) {
      console.log(result.error.message);
    }
  }
}

```



```

        }
    };

    return <>
        <VStack>
            <PaymentElement/>
            <Button colorScheme={'green'} disabled={!stripe} onClick={handleSubmit}/>
        </VStack>
    </>
}

export default IntegratedCheckout

```

This file defines two components, `IntegratedCheckout` and `CheckoutForm`. The `CheckoutForm` defines a simple form with a `PaymentElement` from Stripe which collects customers' payment details and a **Pay** button that triggers a payment collection request.

This component also calls the `useStripe()` and `useElements()` hook to create an instance of the Stripe SDK that you can use to create payment requests. Once you click the **Pay** button, the `stripe.confirmPayment()` method from the Stripe SDK is called that collects the user's payment data from the `elements` instance and sends it to Stripe backend with a success URL to redirect to if the transaction is successful.

The checkout form has been separated from the rest of the page because the `useStripe()` and `useElements()` hooks need to be called from the context of an `Elements` provider, which has been done in the `IntegratedCheckout`'s return statement. If you moved the Stripe hook calls to the `IntegratedCheckout` component directly, they would be outside the scope of the `Elements` provider and hence would not work.

The `IntegratedCheckout` component reuses the `CartItem` and `TotalFooter` components to render the cart items and the total amount. It also renders two input fields to collect the customer's information and an **Initiate payment** button that sends a request to the Java backend server to create the client secret key using the customer and cart details. Once the client secret key is received, the `CheckoutForm` is rendered, which handles the collection of the payment details from the customer.

Apart from that, `useEffect` is used to call the `loadStripe` method. This effect is run only once when the component renders so that the Stripe SDK isn't loaded multiple times when the component's internal states are updated.

To run the code above, you will also need to add two new environment variables to your frontend project: `VITE_STRIPE_API_KEY` and `VITE_CLIENT_BASE_URL`. The Stripe API key variable will hold the publishable API key from the [Stripe dashboard](#), and the client base URL variable will contain the link to the client app (which is the frontend app itself) so that it can be passed to the Stripe SDK for handling success and failure redirects.

To do that, add the following code to your `.env` file in the frontend directory:

```
VITE_STRIPE_API_KEY=pk_test_xxxxxxxxxx # Your key here
VITE_CLIENT_BASE_URL=http://localhost:5173
```

Finally, update the `App.tsx` file to include the `IntegratedCheckout` component at the `/integrated-checkout` route of the frontend app. Add the following code in the array passed to the `createBrowserRouter` call in the `App` component:

```
{
  path: '/integrated-checkout',
  element: (
    <IntegratedCheckout/>
  )
},
```

This completes the setup needed on the frontend. Next, create a new route on your backend server that creates the client secret key needed to handle integrated checkout sessions on

your frontend app.

Building the Backend

To ensure that the frontend integration is not abused by attackers (since frontend code is easier to crack than backend), Stripe requires you to generate a unique client secret on your backend server and verifies each integrated payment request with the client secret generated on the backend to make sure that it is indeed your app that's trying to collect payments. To do that, you need to set up another route in the backend that creates client secrets based on customer and cart information.

For creating the client secret key on your server, create a new method in your `PaymentController` class with the name `integratedCheckout` and save the following code in it:

```
@PostMapping("/checkout/integrated")
String integratedCheckout(@RequestBody RequestDTO requestDTO)

    Stripe.apiKey = STRIPE_API_KEY;

    // Start by finding existing customer or creating a new one
    Customer customer = CustomerUtil.findOrCreateCustomer(requestDTO);

    // Create a PaymentIntent and send it's client secret to frontend
    PaymentIntentCreateParams params =
        PaymentIntentCreateParams.builder()
            .setAmount(Long.parseLong(calculateOrderAmount(requestDTO)))
            .setCurrency("usd")
            .setCustomer(customer.getId())
            .setAutomaticPaymentMethods(
                PaymentIntentCreateParams.AutomaticPaymentMethods.builder()
                    .setEnabled(true)
```

```

        .build()
    )
    .build();

    PaymentIntent paymentIntent = PaymentIntent.create(params);

    // Send the client secret from the payment intent to the client
    return paymentIntent.getClientSecret();
}

```

Similar to how the checkout session was built using a builder class that accepts the configuration for the payment request, the integrated checkout flow requires you to build a payment session with the amount, currency, and payment methods. Unlike the checkout session, you can not associate [line items](#) with a payment session unless you create an invoice, which you will learn in a later section of the tutorial.

Since you are not passing in the line items to the checkout session builder, you need to manually calculate the total amount for the cart items and send the amount to the Stripe backend. Use your `ProductDAO` to find and add the prices for each product in the cart.

To do that, define a new method `calculateOrderAmount` and add the following code in it:

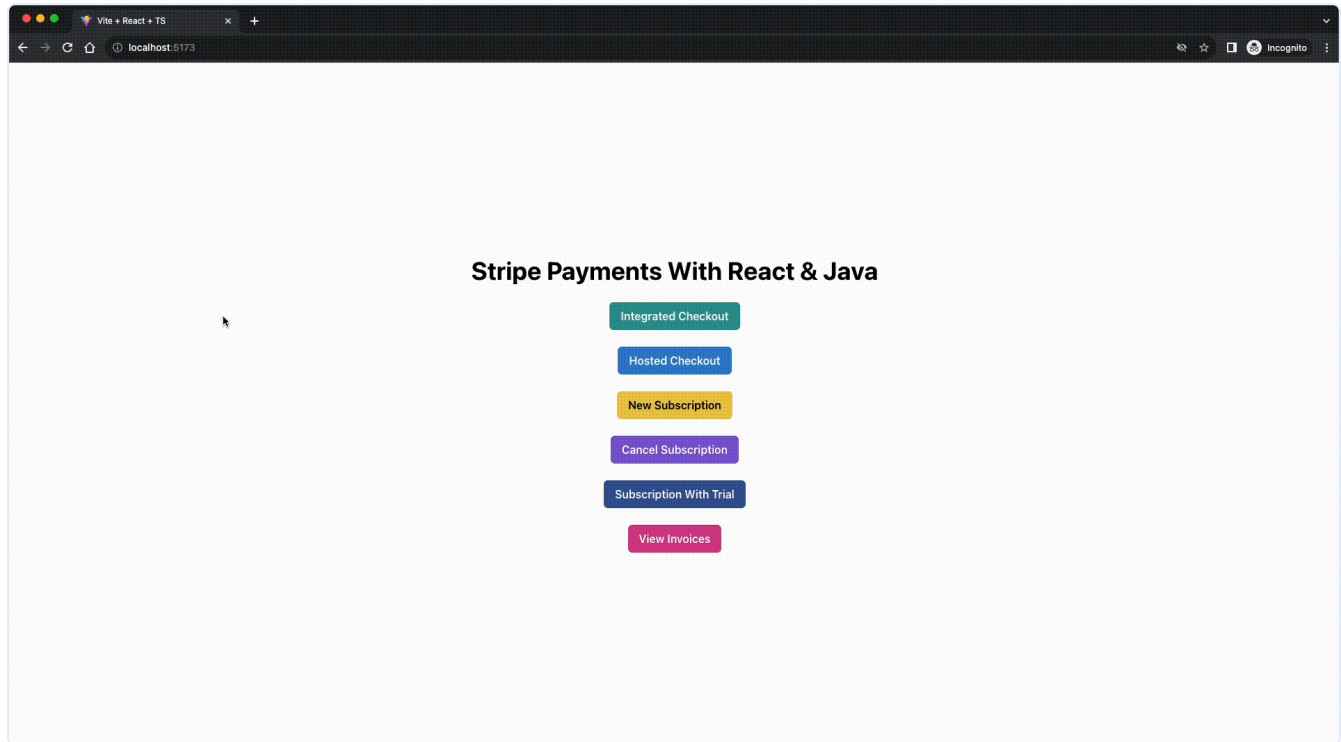
```

static String calculateOrderAmount(Product[] items) {
    long total = 0L;

    for (Product item: items) {
        // Look up the application database to find the price
        total += ProductDAO.getProduct(item.getId()).getDefaultValue();
    }
    return String.valueOf(total);
}

```

That should be sufficient to set up the integrated checkout flow on both the frontend and the backend. You can restart the development servers for the server and client and try out the new integrated checkout flow in the frontend app. Here's what the integrated flow will look like:



— An integrated checkout flow.

This completes a basic integrated checkout flow in your app. You can now further explore the Stripe documentation to [customize the payment methods](#) or integrate more components to help you with other operations such as [address collection](#), [payment requests](#), [link integration](#), and more!

Setting Up Subscriptions for Recurring Services

A common offering from online stores nowadays is a subscription. Whether you are building a marketplace for services or offering a digital product periodically, a subscription is the perfect

solution for giving your customers periodic access to your service for a small fee compared to a one-time purchase.

Stripe can help you set up and cancel subscriptions easily. You can also offer free trials as part of your subscription so that users may try your offering before committing to it.

Setting Up a New Subscription

Setting up a new subscription is straightforward using the hosted checkout flow. You will only need to change a few parameters when building the checkout request and create a new page (by reusing the existing components) to show a checkout page for a new subscription. To start, create a **NewSubscription.tsx** file in the frontend **components** folder. Paste the following code in it:

```
import {Center, Heading, VStack} from "@chakra-ui/react";
import {useState} from "react";
import CartItem, {ItemData} from "../components/CartItem.tsx";
import TotalFooter from "../components/TotalFooter.tsx";
import CustomerDetails from "../components/CustomerDetails.tsx";
import {Subscriptions} from "../data.ts";

function NewSubscription() {
  const [items] = useState<ItemData[]>(Subscriptions)
  return <>
    <Center h={'100vh'} color='black'>
      <VStack spacing='24px'>
        <Heading>New Subscription Example</Heading>
        {items.map(elem => {
          return <CartItem data={elem} mode={'subscription':
        }}}
        <TotalFooter total={4.99} mode={"subscription"}/>
        <CustomerDetails data={items} endpoint={"/subscrip
      </VStack>
    </Center>
```

```

        </>
    }

    export default NewSubscription

```

In the code above, the cart data is taken from the **data.ts** file, and it only contains one item to simplify the process. In real-world scenarios, you can have multiple items as part of one subscription order.

To render this component on the right route, add the following code in the array passed to the `createBrowserRouter` call in the **App.tsx** component:

```

{
    path: '/new-subscription',
    element: (
        <NewSubscription/>
    )
},

```

This completes the setup needed on the frontend. On the backend, create a new route `/subscription/new` to create a new hosted checkout session for a subscription product. Create a `newSubscription` method in the **backend/src/main/java/com/kinsta/stripejava/backend** directory and save the following code in it:

```

@PostMapping("/subscriptions/new")
String newSubscription(@RequestBody RequestDTO requestDTO) thi

```

```

Stripe.apiKey = STRIPE_API_KEY;

String clientBaseURL = System.getenv().get("CLIENT_BASE_URL");

// Start by finding existing customer record from Stripe
Customer customer = CustomerUtil.findOrCreateCustomer(request);

// Next, create a checkout session by adding the details
SessionCreateParams.Builder paramsBuilder =
    SessionCreateParams.builder()
        // For subscriptions, you need to set the
        .setMode(SessionCreateParams.Mode.SUBSCRIPTIONS)
        .setCustomer(customer.getId())
        .setSuccessUrl(clientBaseURL + "/success?session_id=")
        .setCancelUrl(clientBaseURL + "/failure");

for (Product product : requestDTO.getItems()) {
    paramsBuilder.addLineItem(
        SessionCreateParams.LineItem.builder()
            .setQuantity(1L)
            .setPriceData(
                PriceData.builder()
                    .setProductData(
                        PriceData.ProductData.builder()
                            .putMetadataKey("product_id", product.getId())
                            .setName(product.getName())
                            .build()
                    )
                    .setCurrency(ProductData.getCurrency())
                    .setUnitAmountDecimal(product.getUnitAmountDecimal())
                    // For subscriptions,
                    .setRecurring(PriceData.Recurring.builder())
                        .build());
            .build());
}

Session session = Session.create(paramsBuilder.build());

```



```
        return session.getUrl();  
    }
```

The code in this method is quite similar to the code in the `hostedCheckout` method, except that the mode set for creating the session is `subscription` instead of `product` and before creating the session, a value is set for the recurrence interval for the subscription.

This instructs Stripe to treat this checkout as a subscription checkout instead of a one-time payment. Similar to the `hostedCheckout` method, this method also returns the URL of the hosted checkout page as the HTTP response to the client. The client is set to redirect to the URL received, allowing the customer to complete the payment.

You can restart the development servers for both the client and the server and see the new subscription page in action. Here's what it looks like:



— A hosted subscription checkout flow.

Canceling an Existing Subscription

Now that you know how to create new subscriptions, let's learn how to enable your customers to cancel existing subscriptions. Since the demo app built in this tutorial does not contain any authentication setup, use a form to allow the customer to enter their email to look up their subscriptions and then provide each subscription item with a cancel button to allow the user to cancel it.

To do this, you will need to do the following:

1. Update the `CartItem` component to show a cancel button on the cancel subscriptions page.
2. Create a `CancelSubscription` component that first shows an input field and a button for the customer to search subscriptions using their email address and then renders a list of subscriptions using the updated `CartItem` component.
3. Create a new method in the backend server that can look up subscriptions from the Stripe backend using the customer's email address.
4. Create a new method in the backend server that can cancel a subscription based on the subscription ID passed to it.

Start by updating the `CartItem` component to make it look like this:

```
// Existing imports here

function CartItem(props: CartItemProps) {

  // Add this hook call and the cancelSubscription method to cancel
  const toast = useToast()
  const cancelSubscription = () => {

    fetch(process.env.VITE_SERVER_BASE_URL + "/subscriptions/cancel", {
      method: "POST",
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
```

```

        subscriptionId: props.data.stripeSubscriptionData'
      })
    })
    .then(r => r.text())
    .then(() => {
      toast({
        title: 'Subscription cancelled.',
        description: "We've cancelled your subscription",
        status: 'success',
        duration: 9000,
        isClosable: true,
      })

      if (props.onCancelled)
        props.onCancelled()
    })
  }

  return <Card direction={{base: 'column', sm: 'row'}}
    overflow='hidden'
    width={'xl'}
    variant='outline'>
    <Image
      objectFit='cover'
      maxW={{base: '100%', sm: '200px'}}
      src={props.data.image}
    />
    <Stack mt='6' spacing='3'>
      <CardBody>
        <VStack spacing={'3'} alignItems={"flex-start"}>
          <Heading size='md'>{props.data.name}</Heading>
          <VStack spacing={'1'} alignItems={"flex-start"}>
            <Text>
              {props.data.description}
            </Text>
            {(props.mode === "checkout" ? <Text>
              {"Quantity: " + props.data.quantity}
            </Text> : <></>)}
          </VStack>
        </CardBody>
      </Stack>
    </Card>
  )
}

```

```

        { /* <----- Add this block ----->
        {(props.mode === "subscription" && props.data
        <VStack spacing={'1'} alignItems={"flex-start">
            <Text>
                {"Next Payment Date: " + props.data
            </Text>
            <Text>
                {"Subscribed On: " + props.data.st
            </Text>
            {(props.data.stripeSubscriptionData.tr
                {"Free Trial Running Until: " + p
            </Text> : <></>}}
        </VStack> : <></>)}
    </VStack>

</CardBody>

<CardFooter>
    <VStack alignItems={'flex-start'}>
        <Text color='blue.600' fontSize='2xl'>
            {"$" + props.data.price}
        </Text>
        { /* <----- Add this block ----->
        {(props.data.stripeSubscriptionData ?
            <Button colorScheme='red' onClick={cancel
            : <></>)}
        </VStack>
    </CardFooter>
</Stack>
</Card>
}

// Existing types here

export default CartItem

```

Next, create a **CancelSubscription.tsx** component in your frontend's **routes** directory and save the following code in it:

```
import {Button, Center, Heading, Input, VStack} from "@chakra-ui/react";
import {useState} from "react";
import CartItem, {ItemData, ServerSubscriptionsResponseType} from "../data.ts";
import {Subscriptions} from "../data.ts";

function CancelSubscription() {
  const [email, setEmail] = useState("")
  const [subscriptions, setSubscriptions] = useState<ItemData[]>([])

  const onCustomerEmailChange = (ev: React.ChangeEvent<HTMLInputElement>) => {
    setEmail(ev.target.value)
  }

  const listSubscriptions = () => {
    fetch(process.env.VITE_SERVER_BASE_URL + "/subscriptions/", {
      method: "POST",
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        customerEmail: email,
      })
    })
      .then(r => r.json())
      .then((r: ServerSubscriptionsResponseType[]) => {

        const subscriptionsList: ItemData[] = []

        r.forEach(subscriptionItem => {

          let subscriptionDetails = Subscriptions.find(
            (subscription) => subscription.email === subscriptionItem.email
          )

          if (subscriptionDetails) {
            subscriptionsList.push(subscriptionDetails)
          }
        })
      })
      setSubscriptions(subscriptionsList)
    }

  return (
    <VStack>
      <Heading>Cancel Subscription</Heading>
      <Input type="text" value={email} onChange={onCustomerEmailChange}/>
      <Button onClick={listSubscriptions}>Cancel Subscription</Button>
    </VStack>
  )
}
```

```

        subscriptionDetails = {
            ...subscriptionDetails,
            price: Number.parseInt(subscriptionItem.price),
            stripeSubscriptionData: subscriptionItem.stripeSubscriptionData
        }

        subscriptionsList.push(subscriptionDetails)
    } else {
        console.log("Item not found!")
    }
})

setSubscriptions(subscriptionsList)
})

}

const removeSubscription = (id: string | undefined) => {
    const newSubscriptionsList = subscriptions.filter(elem => elem.id !== id)
    setSubscriptions(newSubscriptionsList)
}

return <>
    <Center h={'100vh'} color='black'>
        <VStack spacing={3} width={'xl'}>
            <Heading>Cancel Subscription Example</Heading>
            {(subscriptions.length === 0 ? <>
                <Input variant='filled' placeholder='Customer email'
                    value={email}/>
                <Button onClick={listSubscriptions} colorScheme='gray'>Cancel</Button>
            </> : <></>)}
            {subscriptions.map(elem => {
                return <CartItem data={elem} mode={'subscription'} />
            })}
        </VStack>
    </Center>
</>
}

```

```
export default CancelSubscription
```

This component renders an input field and a button for customers to enter their email and start looking for subscriptions. If subscriptions are found, the input field and button are hidden, and a list of subscriptions is displayed on the screen. For each subscription item, the component passes a `removeSubscription` method that requests the Java backend server to cancel the subscription on the Stripe backend.

To attach it to the `/cancel-subscription` route on your frontend app, add the following code in the array passed to the `createBrowserRouter` call in the `App` component:

```
{
  path: '/cancel-subscription',
  element: (
    <CancelSubscription/>
  )
},
```

To search for subscriptions on the backend server, add a `viewSubscriptions` method in the `PaymentController` class of your backend project with the following contents:

```
@PostMapping("/subscriptions/list")
List<Map<String, String>> viewSubscriptions(@RequestBody Request

    Stripe.apiKey = STRIPE_API_KEY;

    // Start by finding existing customer record from Stripe
```

```

Customer customer = CustomerUtil.findCustomerByEmail(request.getEmail());

// If no customer record was found, no subscriptions exist
if (customer == null) {
    return new ArrayList<>();
}

// Search for subscriptions for the current customer
SubscriptionCollection subscriptions = Subscription.list(
    SubscriptionListParams.builder()
        .setCustomer(customer.getId())
        .build());

List<Map<String, String>> response = new ArrayList<>();

// For each subscription record, query its item records and
for (Subscription subscription : subscriptions.getData())
    SubscriptionItemCollection currSubscriptionItems =
        SubscriptionItem.list(SubscriptionItemListParams.builder()
            .setSubscription(subscription.getId())
            .addExpand("data.price.product")
            .build());

    for (SubscriptionItem item : currSubscriptionItems.getData())
        HashMap<String, String> subscriptionData = new HashMap<>();
        subscriptionData.put("appProductId", item.getPrice().getAppProductId());
        subscriptionData.put("subscriptionId", subscription.getId());
        subscriptionData.put("subscribedOn", new SimpleDateFormat("dd/MM/yyyy").format(subscription.getSubscribedOn()));
        subscriptionData.put("nextPaymentDate", new SimpleDateFormat("dd/MM/yyyy").format(subscription.getNextPaymentDate()));
        subscriptionData.put("price", item.getPrice().getPrice());

        if (subscription.getTrialEnd() != null && new Date().before(subscription.getTrialEnd()))
            subscriptionData.put("trialEndsOn", new SimpleDateFormat("dd/MM/yyyy").format(subscription.getTrialEnd()));

        response.add(subscriptionData);
    }

}

return response;

```



```
}
```

The method above first finds the customer object for the given user in Stripe. Then, it searches for active subscriptions of the customer. Once the list of subscriptions is received, it extracts the items from them and finds the corresponding products in the app product database to send to the frontend. This is important because the ID with which the frontend identifies each product in the app database may or may not be the same as the product ID stored in Stripe.

Finally, create a `cancelSubscription` method in the `PaymentController` class and paste the code below to delete a subscription based on the subscription ID passed.

```
@PostMapping("/subscriptions/cancel")
String cancelSubscription(@RequestBody RequestDTO requestDTO)
    Stripe.apiKey = STRIPE_API_KEY;

    Subscription subscription =
        Subscription.retrieve(
            requestDTO.getSubscriptionId()
        );

    Subscription deletedSubscription =
        subscription.cancel();

    return deletedSubscription.getStatus();
}
```

This method retrieves the subscription object from Stripe, calls the cancel method on it, and then returns the subscription status to the client. However, to be able to run this, you need to update your DTO object to add the `subscriptionId` field. Do that by adding the following

field and method in the RequestDTO class:

```
package com.kinsta.stripejava.backend;

import com.stripe.model.Product;

public class RequestDTO {
    // ... other fields ...

    // Add this
    String subscriptionId;

    // ... other getters ...

    // Add this
    public String getSubscriptionId() {
        return subscriptionId;
    }
}
```

Once you add this, you can now re-run the development server for both the backend and the frontend app and see the cancel flow in action:

Stripe Payments With React & Java

Integrated Checkout

Hosted Checkout

New Subscription

Cancel Subscription

Subscription With Trial

View Invoices

— A subscription cancel flow.

Setting Up Free Trials for Subscriptions With Zero-Value Transactions

A common feature with most modern subscriptions is to offer a short free trial period before charging the user. This allows users to explore the product or service without investing in it. However, it is best to store the customer's payment details while signing them up for the free trial so that you can easily charge them as soon as the trial ends.

Stripe greatly simplifies the creation of such subscriptions. To begin, generate a new component within the **frontend/routes** directory named **SubscriptionWithTrial.tsx**, and paste the following code:

```
import {Center, Heading, VStack} from "@chakra-ui/react";  
import {useState} from "react";
```

```

import CartItem, {ItemData} from "../components/CartItem.tsx";
import TotalFooter from "../components/TotalFooter.tsx";
import CustomerDetails from "../components/CustomerDetails.tsx";
import {Subscriptions} from "../data.ts";

function SubscriptionWithTrial() {
  const [items] = useState<ItemData[]>(Subscriptions)
  return <>
    <Center h={'100vh'} color='black'>
      <VStack spacing='24px'>
        <Heading>New Subscription With Trial Example</Heading>
        {items.map(elem => {
          return <CartItem data={elem} mode={'subscription'} />
        })}
        <TotalFooter total={4.99} mode={"trial"} />
        <CustomerDetails data={items} endpoint={"/subscriptions"} />
      </VStack>
    </Center>
  </>
}

export default SubscriptionWithTrial

```

This component reuses the components created earlier. The key difference between this and the `NewSubscription` component is that it passes the mode for `TotalFooter` as **trial** instead of **subscription**. This makes the `TotalFooter` component render a text saying that the customer can start the free trial now but will be charged after a month.

To attach this component to the `/subscription-with-trial` route on your frontend app, add the following code in the array passed to the `createBrowserRouter` call in the `App` component:

```

{
    path: '/subscription-with-trial',
    element: (
        <SubscriptionWithTrial/>
    )
},

```

To build the checkout flow for subscriptions with **trial** on the backend, create a new method named `newSubscriptionWithTrial` in the `PaymentController` class and add the following code:

```

@PostMapping("/subscriptions/trial")
String newSubscriptionWithTrial(@RequestBody RequestDTO request) {

    Stripe.apiKey = STRIPE_API_KEY;

    String clientBaseUrl = System.getenv().get("CLIENT_BASE_URL");

    // Start by finding existing customer record from Stripe
    Customer customer = CustomerUtil.findOrCreateCustomer(request);

    // Next, create a checkout session by adding the details of the subscription
    SessionCreateParams.Builder paramsBuilder =
        SessionCreateParams.builder()
            .setMode(SessionCreateParams.Mode.SUBSCRIPTION)
            .setCustomer(customer.getId())
            .setSuccessUrl(clientBaseUrl + "/success?session_id={CHECKOUT_SESSION_ID}")
            .setCancelUrl(clientBaseUrl + "/failure")
            // For trials, you need to set the trial period
            .setSubscriptionData(SessionCreateParams.SubscriptionData.builder()

```

```

        for (Product product : requestDTO.getItems()) {
            paramsBuilder.addLineItem(
                SessionCreateParams.LineItem.builder()
                    .setQuantity(1L)
                    .setPriceData(
                        PriceData.builder()
                            .setProductData(
                                PriceData.ProductData.builder()
                                    .putMetadata("trialPeriod", 30)
                                    .setName(product.getName())
                                    .build()
                            )
                            .setCurrency(ProductData.getCurrency())
                            .setUnitAmountDecimal(product.getPrice())
                            .setRecurring(PriceData.RecurringData.builder()
                                .build())
                            .build())
                    .build());
        }

        Session session = Session.create(paramsBuilder.build());

        return session.getUrl();
    }

```

This code is quite similar to that of `newSubscription` method. The only (and the most important) difference is that a trial period is passed to the session create parameters object with the value of 30, indicating a free trial period of 30 days.

You can now save the changes and re-run the development server for the backend and the frontend to see the subscription with free trial workflow in action:

Stripe Payments With React & Java

Integrated Checkout

Hosted Checkout

New Subscription

Cancel Subscription

Subscription With Trial

View Invoices

— A subscription with free trial flow.

Generating Invoices for Payments

For subscriptions, Stripe automatically generates invoices for each payment, even if it is a zero value transaction for trial signup. For one-off payments, you can choose to create invoices if needed.

To start associating all payments with invoices, update the body of the payload being sent in the `initiatePayment` function of the `CustomerDetails` component in the frontend app to contain the following property:

```
invoiceNeeded: true
```

You will also need to add this property in the body of the payload being sent to the server in the `createTransactionSecret` function of the `IntegratedCheckout` component.

Next, update the backend routes to check for this new property and update the Stripe SDK calls accordingly.

For the hosted checkout method, to add the invoicing functionality, update the `hostedCheckout` method by adding the following lines of code:

```
@PostMapping("/checkout/hosted")
String hostedCheckout(@RequestBody RequestDTO requestDTO) throws IOException {
    // ... other operations being done after creating the Session

    // Add the following block of code just before the Session creation
    if (requestDTO.isInvoiceNeeded()) {
        paramsBuilder.setInvoiceCreation(SessionCreateParams.INVOICE_CREATION);
    }

    Session session = Session.create(paramsBuilder.build());

    return session.getUrl();
}
```

This will check for the `invoiceNeeded` field and set the create parameters accordingly.

Adding an invoice to an integrated payment is slightly tricky. You can not simply set a parameter to instruct Stripe to create an invoice with the payment automatically. You must manually create the invoice and then a linked payment intent.

If the payment intent is successfully paid and completed, the invoice is marked as paid; otherwise, the invoice remains unpaid. While this makes logical sense, it can be a little complex to implement (especially when there are no clear examples or references to follow).

To implement this, update the `integratedCheckout` method to make it look like this:

```
String integratedCheckout(@RequestBody RequestDTO requestDTO) throws StripeException {

    Stripe.apiKey = STRIPE_API_KEY;

    // Start by finding an existing customer or creating a new one
    Customer customer = CustomerUtil.findOrCreateCustomer(requestDTO);

    PaymentIntent paymentIntent;

    if (!requestDTO.isInvoiceNeeded()) {
        // If the invoice is not needed, create a PaymentIntent
        PaymentIntentCreateParams params =
            PaymentIntentCreateParams.builder()
                .setAmount(Long.parseLong(calculateOrGetAmount(requestDTO)))
                .setCurrency("usd")
                .setCustomer(customer.getId())
                .setAutomaticPaymentMethods(
                    PaymentIntentCreateParams.AutomaticPaymentMethods
                        .builder()
                        .setEnabled(true)
                        .build()
                )
                .build();

        paymentIntent = PaymentIntent.create(params);
    } else {
        // If invoice is needed, create the invoice object, and
        InvoiceCreateParams invoiceCreateParams = new InvoiceCreateParams()
            .setCustomer(customer.getId())
            .build();

        Invoice invoice = Invoice.create(invoiceCreateParams);

        // Add each item to the invoice one by one
    }
}
```

```

for (Product product : requestDTO.getItems()) {

    // Look for existing Product in Stripe before creating
    Product stripeProduct;

    ProductSearchResult results = Product.search(ProductSearchResult
        .setQuery("metadata['app_id']:" + product.getId())
        .build());

    if (results.getData().size() != 0)
        stripeProduct = results.getData().get(0);
    else {

        // If a product is not found in Stripe database
        ProductCreateParams productCreateParams = new ProductCreateParams()
            .setName(product.getName())
            .putMetadata("app_id", product.getId())
            .build();

        stripeProduct = Product.create(productCreateParams);
    }

    // Create an invoice line item using the product created
    InvoiceItemCreateParams invoiceItemCreateParams = new InvoiceItemCreateParams()
        .setInvoice(invoice.getId())
        .setQuantity(1L)
        .setCustomer(customer.getId())
        .setPriceData(
            InvoiceItemCreateParams.PriceData
                .setProduct(stripeProduct.getId())
                .setCurrency(ProductDAO.getCurrency())
                .setUnitAmountDecimal(ProductDAO.getPrice())
                .build())
        .build();

    InvoiceItem.create(invoiceItemCreateParams);
}

// Mark the invoice as final so that a PaymentIntent can be created

```

```

        invoice = invoice.finalizeInvoice();

        // Retrieve the payment intent object from the invoice
        paymentIntent = PaymentIntent.retrieve(invoice.getPaymentIntent());
    }

    // Send the client secret from the payment intent to the client
    return paymentIntent.getClientSecret();
}

```

The old code of this method is moved into the `if` block that checks if the `invoiceNeeded` field is `false`. If it is found true, the method now creates an invoice with invoice items and marks it as finalized so that it can be paid.

Then, it retrieves the payment intent automatically created when the invoice was finalized and sends the client secret from this payment intent to the client. Once the client completes the integrated checkout flow, the payment is collected, and the invoice is marked as paid.

This completes the setup needed to start generating invoices from your application. You can head over to [the invoices section](#) on your Stripe dashboard to look at the invoices your app generates with each purchase and subscription payment.

However, Stripe also allows you to access the invoices over its API to make a self-service experience for customers to download invoices whenever they want.

To do that, create a new component in the **frontend/routes** directory named **ViewInvoices.tsx**. Paste the following code in it:

```

import {Button, Card, Center, Heading, HStack, IconButton, Input,
import {useState} from "react";
import {DownloadIcon} from "@chakra-ui/icons";

```

```

function ViewInvoices() {
  const [email, setEmail] = useState("")
  const [invoices, setInvoices] = useState<InvoiceData[]>([])

  const onCustomerEmailChange = (ev: React.ChangeEvent<HTMLInput
    setEmail(ev.target.value)
  }

  const listInvoices = () => {

    fetch(process.env.VITE_SERVER_BASE_URL + "/invoices/list",
      method: "POST",
      headers: {'Content-Type': 'application/json'},
      body: JSON.stringify({
        customerEmail: email,
      })
    )
    .then(r => r.json())
    .then((r: InvoiceData[]) => {
      setInvoices(r)
    })
  }

  return <>
    <Center h={'100vh'} color='black'>
      <VStack spacing={3} width={'xl'}>
        <Heading>View Invoices for Customer</Heading>
        {(invoices.length === 0 ? <>
          <Input variant='filled' placeholder='Customer
            value={email}/>
          <Button onClick={listInvoices} colorScheme={'
        </> : <></>)}
        {invoices.map(elem => {
          return <Card direction={{base: 'column', sm:
            overflow='hidden'
            alignItems={'center'}
            justifyContent={'space-between'}
            padding={'8px'}

```

```

        width={500}
        variant='outline'>
      <Text>
        {elem.number}
      </Text>
      <HStack spacing={"3"}>
        <Text color='blue.600' fontSize='2xl'>
          {"$" + elem.amount}
        </Text>
        <IconButton onClick={() => {
          window.location.href = elem.url
        }} icon={<DownloadIcon/>} aria-label=
      </HStack>
    </Card>
  )}
</VStack>
</Center>
</>
}

interface InvoiceData {
  number: string,
  amount: string,
  url: string
}

export default ViewInvoices

```

Similar to the `CancelSubscription` component, this component displays an input field for the customer to enter their email and a button to search for invoices. Once invoices are found, the input field and button are hidden, and a list of invoices with the invoice number, total amount, and a button to download the invoice PDF is shown to the customer.

To implement the backend method that searches for invoices of the given customer and sends back the relevant information (invoice number, amount, and PDF URL), add the following method in your `PaymentController` class on the backend;

```

@PostMapping("/invoices/list")
List<Map<String, String>> listInvoices(@RequestBody RequestDTO requestDTO) {

    Stripe.apiKey = STRIPE_API_KEY;

    // Start by finding existing customer record from Stripe
    Customer customer = CustomerUtil.findCustomerByEmail(requestDTO.getEmail());

    // If no customer record was found, no subscriptions exist
    if (customer == null) {
        return new ArrayList<>();
    }

    // Search for invoices for the current customer
    Map<String, Object> invoiceSearchParams = new HashMap<>();
    invoiceSearchParams.put("customer", customer.getId());
    InvoiceCollection invoices =
        Invoice.list(invoiceSearchParams);

    List<Map<String, String>> response = new ArrayList<>();

    // For each invoice, extract its number, amount, and PDF URL
    for (Invoice invoice : invoices.getData()) {
        HashMap<String, String> map = new HashMap<>();

        map.put("number", invoice.getNumber());
        map.put("amount", String.valueOf(invoice.getTotal()));
        map.put("url", invoice.getInvoicePdf());

        response.add(map);
    }

    return response;
}

```

The method first looks for the customer by the email address provided to it. Then, it looks for invoices of this customer that are marked as paid. Once the list of invoices is found, it extracts

the invoice number, amount, and PDF URL and sends back a list of this information to the client app.

This is how the invoices flow looks like:



— Viewing invoices

This completes the development of our demo Java app ([frontend](#) & [backend](#)). In the next section, you will learn how to deploy this app to Kinsta so you can access it online.

Deploying Your App to Kinsta

Once your application is ready, you can deploy it to Kinsta. Kinsta supports deployments from your preferred Git provider ([Bitbucket](#), [GitHub](#), or [GitLab](#)). Connect your app's source code repositories to Kinsta, it automatically deploys your app whenever there's a change in the code.

Prepare Your Projects

To deploy your apps to production, identify the build and deploy commands that Kinsta will use. For frontend, make sure that your **package.json** file has the following scripts defined in it:

```
"scripts": {
  "dev": "vite",
  "build": "NODE_ENV=production tsc && vite build",
  "start": "serve ./dist",
  "lint": "eslint . --ext ts,tsx --report-unused-disable-directives",
  "preview": "vite preview"
},
```

You will also need to install the [serve](#) npm package that allows you to serve static websites. This package will be used to serve the production build of your app from the Kinsta deployment environment. You can install it by running the following command:

```
npm i serve
```

Once you build your app using vite, the entire app will be packaged into a single file, **index.html**, as the configuration of React that you are using in this tutorial is meant to create Single Page Applications. While this does not make a huge difference to your users, you need to set up some extra configuration to handle browser routing and navigation in such apps.

With the current configuration, you can only access your app at the base URL of your deployment. If the base URL of the deployment is **example.com**, any requests to **example.com/some-route** will lead to HTTP 404 errors.

This is because your server only has one file to serve, the **index.html** file. A request sent to **example.com/some-route** will start looking for the file **some-route/index.html**, which does not exist; hence it will receive a 404 Not Found response.

To fix this, create a file named **serve.json** in your **frontend/public** folder and save the following code in it:

```
{
  "rewrites": [
    { "source": "**", "destination": "/index.html" }
  ]
}
```

This file will instruct serve to rewrite all incoming requests to route to the **index.html** file while still showing the path that the original request was sent to in the response. This will help you correctly serve your app's success and failure pages when Stripe redirects your customers back to your app.

For the backend, create a [Dockerfile](#) to set up just the right environment for your Java application. Using a Dockerfile ensures that the environment provided to your Java app is the same across all hosts (be it your local development host or the Kinsta deployment host) and you can make sure that your app runs as expected.

To do this, create a file named **Dockerfile** in the **backend** folder and save the following contents in it:

```
FROM openjdk:22-oraclelinux8

LABEL maintainer="krharsh17"
```

```
WORKDIR /app

COPY . /app

RUN ./mvnw clean package

EXPOSE 8080

ENTRYPOINT ["java", "-jar", "/app/target/backend.jar"]
```

This file instructs the runtime to use the [OpenJDK](#) Java image as the base for the deployment container, run the `./mvnw clean package` command to build your app's **JAR** file and use the `java -jar <jar-file>` command to execute it. This completes the preparation of the source code for deployment to Kinsta.

Set Up GitHub Repositories

To get started with deploying the apps, create two GitHub repositories to host your apps' source code. If you use the GitHub CLI, you can do it via the terminal by running the following commands:

```
# Run these in the backend folder
gh repo create stripe-payments-java-react-backend --public --source
git init
git add .
git commit -m "Initial commit"
git push origin main

# Run these in the frontend folder
gh repo create stripe-payments-java-react-frontend --public --source
```

```
git init
git add .
git commit -m "Initial commit"
git push origin main
```

This should create new GitHub repositories in your account and push your apps' code to them. You should be able to access the frontend and backend repositories. Next, deploy these repositories to Kinsta by following these steps:

1. Log in to or create your Kinsta account on the [MyKinsta](#) dashboard.
2. On the left sidebar, click **Applications** and then click **Add Application**.
3. In the modal that appears, choose the repository you want to deploy. If you have multiple branches, you can select the desired branch and give a name to your application.
4. Select one of the available data center locations from the list of 25 options. Kinsta automatically detects the start command for your application.

Remember that you need to provide both your frontend and backend apps with some [environment variables](#) for them to work correctly. The frontend application needs the following environment variables:

- VITE_STRIPE_API_KEY
- VITE_SERVER_BASE_URL
- VITE_CLIENT_BASE_URL

To deploy the backend application, do exactly what we did for the frontend, but for the **Build environment** step, select the **Use Dockerfile to set up container image** radio button and enter `Dockerfile` as the Dockerfile path for your backend application.

The screenshot shows a modal window titled "Add application" with a close button (X) in the top right corner. Inside the modal, there are four tabs: "Application details", "Build environment" (which is selected and highlighted with a blue underline), "Resources", and "Summary". The "Build environment" tab contains the following content:

- Build environment**
Once the application is created you can amend dockerfile details, buildpacks, connections and environment variables under app settings.
[Learn more](#) ⓘ
- Build resources**
The more resources a build process has, the faster it will run.
Standard build machine (1 CPU / 4 GB RAM) - 0.0236 USD / minute
- Set up container image automatically**
If you don't have a Dockerfile, we can set it up based on your repository.
- Use Dockerfile to set up container image** (selected)
If you have a Dockerfile in your repository, we can use that to set up your application.
Dockerfile path
Dockerfile
Context

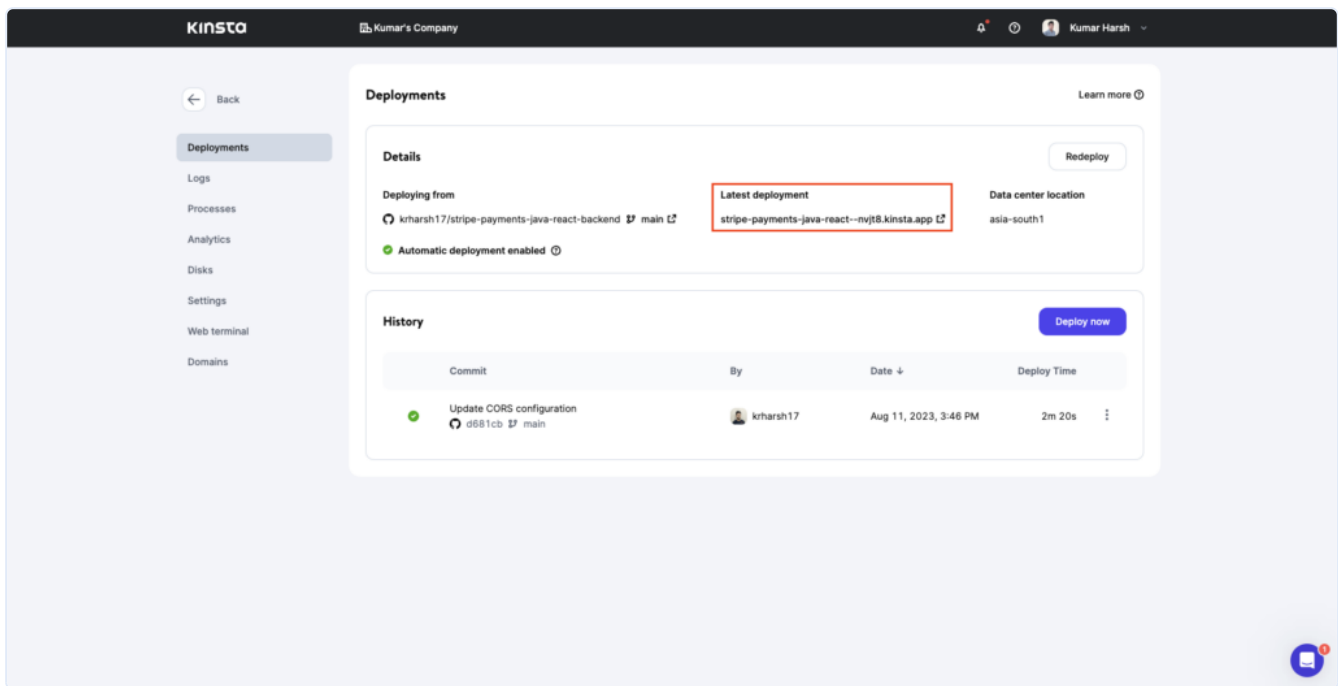
At the bottom of the modal, there are three buttons: "Cancel", "Back", and "Continue". In the bottom right corner, there is a blue circular icon with a white document symbol and a red notification badge with the number "1".

— Setting the build environment details

Remember to add the backend environment variables:

- CLIENT_BASE_URL
- STRIPE_API_KEY

Once the deployment is complete, head over to your applications' details page and access the deployment's URL from there.



— The hosted URL for the apps deployed on Kinsta

Extract the URLs for both the deployed apps. Head over to the [Stripe dashboard](#) to get your secret and publishable API keys.

Ensure to provide the Stripe publishable key to your frontend app (not the secret key). Also, ensure that your base URLs do not have a trailing forward slash (/) at their end. The routes already have leading forward slashes, so having a trailing forward slash at the end of the base URLs will result in two slashes being added to the final URLs.

For your backend app, add the secret key from the Stripe dashboard (not the publishable key). Also, ensure that your client URL does not have a trailing forward slash (/) at the end.

Once the variables are added, go to the application **Deployments** tab and click the **redeploy** button for your backend app. This completes the one-time setup you need to provide your Kinsta deployments with credentials via environment variables.

Moving forward, you can commit changes to your version control. Kinsta will automatically redeploy your application if you ticked the option while deploying; otherwise, you need to trigger re-deployment manually.

Summary

In this article, you have learned how Stripe works and the payment flows that it offers. You have also learned through a detailed example how to integrate Stripe into your Java app to accept one-off payments, set up subscriptions, offer free trials, and generate payment invoices.

Using Stripe and Java together, you can offer a robust payment solution to your customers that can scale well and integrate seamlessly with your existing ecosystem of applications and tools.

Do you use Stripe in your app for collecting payments? If yes, which of the two flows do you prefer—hosted, custom, or in-app? Let us know in the comments below!